

机器学习练习 - 逻辑回归

代码更新地址: <https://github.com/fengdu78/WZU-machine-learning-course>
(<https://github.com/fengdu78/WZU-machine-learning-course>)

代码修改并注释: 黄海广, haiguang2000@wzu.edu.cn (<mailto:haiguang2000@wzu.edu.cn>)

在这一次练习中, 我们将要实现逻辑回归并且应用到一个分类任务。我们还将通过将正则化加入训练算法, 来提高算法的鲁棒性, 并用更复杂的情形来测试它。

逻辑回归

在训练的初始阶段, 我们将要构建一个逻辑回归模型来预测, 某个学生是否被大学录取。设想你是大学相关部分的管理者, 想通过申请学生两次测试的评分, 来决定他们是否被录取。现在你拥有之前申请学生的可以用于训练逻辑回归的训练样本集。对于每一个训练样本, 你有他们两次测试的评分和最后是被录取的结果。为了完成这个预测任务, 我们准备构建一个可以基于两次测试评分来评估录取可能性的分类模型。

让我们从检查数据开始。

```
In [29]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [30]: path = 'ex2data1.txt'
data = pd.read_csv(path, header=None, names=['Exam 1', 'Exam 2', 'Admitted'])
data.head()
```

```
Out[30]:
```

	Exam 1	Exam 2	Admitted
0	34.623660	78.024693	0
1	30.286711	43.894998	0
2	35.847409	72.902198	0
3	60.182599	86.308552	1
4	79.032736	75.344376	1

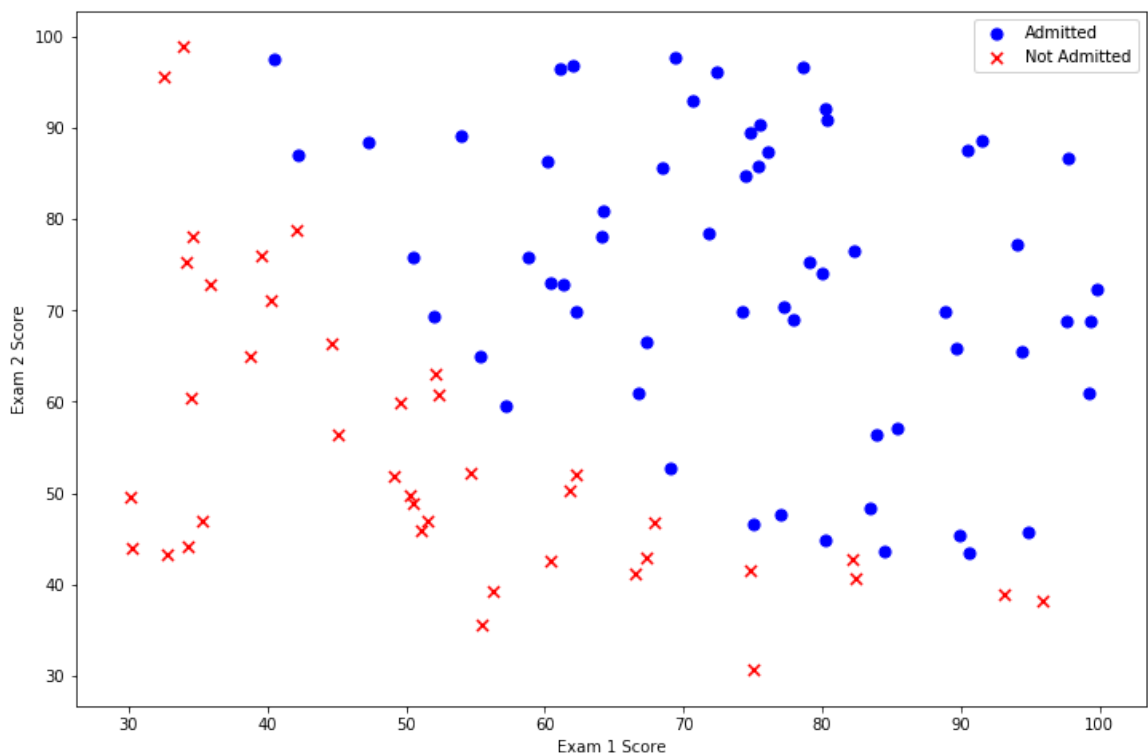
```
In [31]: data.shape
```

```
Out[31]: (100, 3)
```

让我们创建两个分数的散点图, 并使用颜色编码来可视化, 如果样本是正的 (被接纳) 或负的 (未被接纳)。

```
In [32]: positive = data[data['Admitted'].isin([1])]
negative = data[data['Admitted'].isin([0])]

fig, ax = plt.subplots(figsize=(12, 8))
ax.scatter(positive['Exam 1'],
           positive['Exam 2'],
           s=50,
           c='b',
           marker='o',
           label='Admitted')
ax.scatter(negative['Exam 1'],
           negative['Exam 2'],
           s=50,
           c='r',
           marker='x',
           label='Not Admitted')
ax.legend()
ax.set_xlabel('Exam 1 Score')
ax.set_ylabel('Exam 2 Score')
plt.show()
```



看起来在两类间，有一个清晰的决策边界。现在我们需要实现逻辑回归，那样就可以训练一个模型来预测结果。

Sigmoid 函数

g 代表一个常用的逻辑函数 (logistic function) 为 S 形函数 (Sigmoid function)，公式为：

$$g(z) = \frac{1}{1 + e^{-z}}$$

合起来，我们得到逻辑回归模型的假设函数：

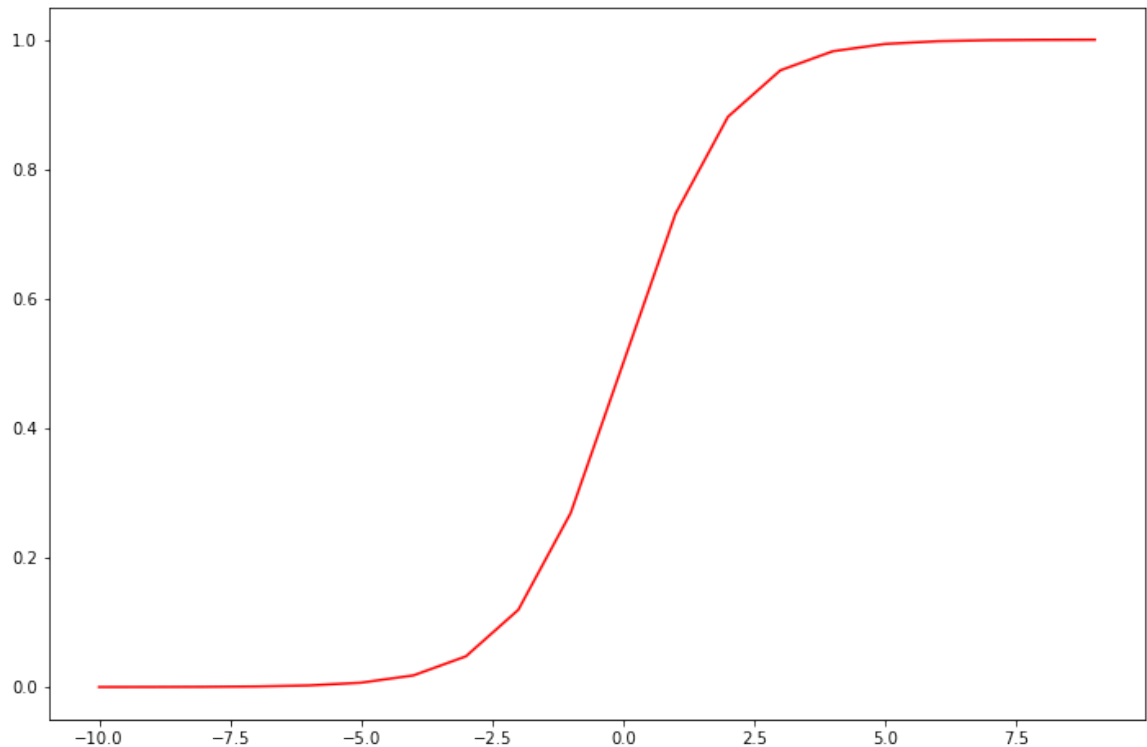
$$h(x) = \frac{1}{1 + e^{-w^T x}}$$

```
In [4]: def sigmoid(z):
        return 1 / (1 + np.exp(-z))
```

让我们做一个快速的检查，来确保它可以工作。

```
In [5]: nums = np.arange(-10, 10, step=1)

fig, ax = plt.subplots(figsize=(12, 8))
ax.plot(nums, sigmoid(nums), 'r')
plt.show()
```



棒极了！现在，我们需要编写代价函数来评估结果。代价函数：

$$J(w) = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})))$$

```
In [6]: def cost(w, X, y):
        w = np.matrix(w)
        X = np.matrix(X)
        y = np.matrix(y)
        first = np.multiply(-y, np.log(sigmoid(X * w.T)))
        second = np.multiply((1 - y), np.log(1 - sigmoid(X * w.T)))
        return np.sum(first - second) / (len(X))
```

现在，我们要做一些设置，和我们在练习1在线性回归的练习很相似。

```
In [7]: # add a ones column - this makes the matrix multiplication work out easier
data.insert(0, 'Ones', 1)

# set X (training data) and y (target variable)
cols = data.shape[1]
X = data.iloc[:, 0:cols - 1]
y = data.iloc[:, cols - 1:cols]

# convert to numpy arrays and initialize the parameter array w
X = np.array(X.values)
y = np.array(y.values)
w = np.zeros(3)
```

让我们来检查矩阵的维度来确保一切良好。

```
In [8]: X.shape, w.shape, y.shape
```

```
Out[8]: ((100, 3), (3,), (100, 1))
```

让我们计算初始化参数的代价函数(w 为0)。

```
In [9]: cost(w, X, y)
```

```
Out[9]: 0.6931471805599453
```

看起来不错，接下来，我们需要一个函数来计算我们的训练数据、标签和一些参数 w 的梯度。

gradient descent(梯度下降)

- 这是批量梯度下降 (batch gradient descent)
- 转化为向量化计算: $\frac{1}{m} X^T (\text{Sigmoid}(XW) - y)$

$$\frac{\partial J(w)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

```
In [10]: def gradient(w, X, y):
w = np.matrix(w)
X = np.matrix(X)
y = np.matrix(y)

parameters = int(w.ravel().shape[1])
grad = np.zeros(parameters)

error = sigmoid(X * w.T) - y

for i in range(parameters):
    term = np.multiply(error, X[:, i])
    grad[i] = np.sum(term) / len(X)

return grad
```

注意，我们实际上没有在这个函数中执行梯度下降，我们仅仅在计算一个梯度步长。在练习中，一个称为“fminunc”的Octave函数是用来优化函数来计算成本和梯度参数。由于我们使用Python，我们可以用SciPy的“optimize”命名空间来做同样的事情。

我们看看用我们的数据和初始参数为0的梯度下降法的结果。

```
In [11]: gradient(w, X, y)
```

```
Out[11]: array([-0.1          , -12.00921659, -11.26284221])
```

现在可以用SciPy's truncated newton (TNC) 实现寻找最优参数。

```
In [12]: import scipy.optimize as opt
result = opt.fmin_tnc(func=cost, x0=w, fprime=gradient, args=(X, y))
result
```

```
Out[12]: (array([-25.16131872,  0.20623159,  0.20147149]), 36, 0)
```

让我们看看在这个结论下代价函数计算结果是个什么样子~

```
In [13]: cost(result[0], X, y)
```

```
Out[13]: 0.20349770158947425
```

预测

接下来，我们需要编写一个函数，用我们所学的参数w来为数据集X输出预测。然后，我们可以使用这个函数来给我们的分类器的训练精度打分。逻辑回归模型的假设函数：

$$h(x) = \frac{1}{1 + e^{-w^T X}}$$

当 h 大于等于0.5时，预测 $y=1$

当 h 小于0.5时，预测 $y=0$ 。

```
In [14]: def predict(w, X):
          probability = sigmoid(X * w.T)
          return [1 if x >= 0.5 else 0 for x in probability]
```

```
In [15]: w_min = np.matrix(result[0])
predictions = predict(w_min, X)
correct = [
    1 if ((a == 1 and b == 1) or (a == 0 and b == 0)) else 0
    for (a, b) in zip(predictions, y)
]
accuracy = (sum(map(int, correct)) % len(correct))
print('accuracy = {0}%'.format(accuracy))
```

```
accuracy = 89%
```

我们的逻辑回归分类器预测正确，如果一个学生被录取或没有录取，达到89%的精确度。不坏！记住，这是训练集的准确性。我们没有保持住了设置或使用交叉验证得到的真实逼近，所以这个数字有可能高于其真实值（这个话题将在以后说明）。

正则化逻辑回归

在训练的第二部分，我们将要通过加入正则项提升逻辑回归算法。如果你对正则化有点眼生，或者喜欢这一节的方程的背景，请参考在"exercises"文件夹中的"ex2.pdf"。简而言之，正则化是成本函数中的一个术语，它使算法更倾向于“更简单”的模型（在这种情况下，模型将更小的系数）。这个理论助于减少过拟合，提高模型的泛化能力。这样，我们开始吧。

设想你是工厂的生产主管，你有一些芯片在两次测试中的测试结果。对于这两次测试，你想决定是否芯片要被接受或抛弃。为了帮助你做出艰难的决定，你拥有过去芯片的测试数据集，从其中你可以构建一个逻辑回归模型。

和第一部分很像，从数据可视化开始吧！

In [16]:

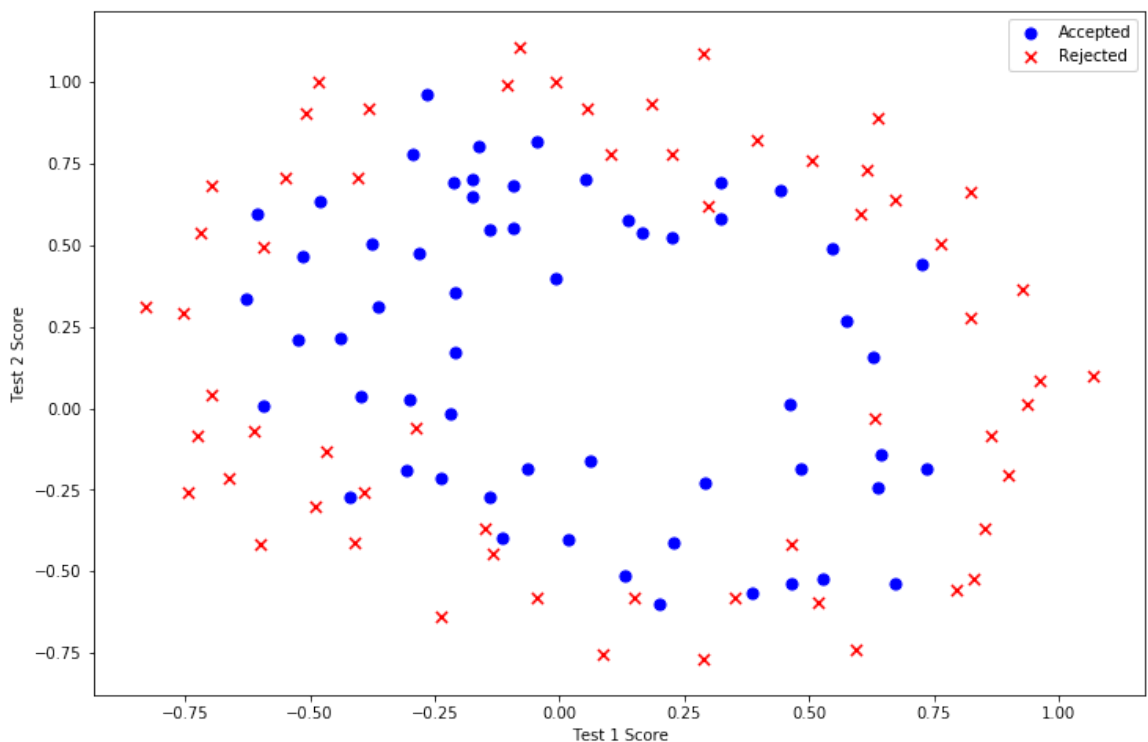
```
path = 'ex2data2.txt'
data2 = pd.read_csv(path, header=None, names=['Test 1', 'Test 2', 'Accepted'])
data2.head()
```

Out[16]:

	Test 1	Test 2	Accepted
0	0.051267	0.69956	1
1	-0.092742	0.68494	1
2	-0.213710	0.69225	1
3	-0.375000	0.50219	1
4	-0.513250	0.46564	1

```
In [17]: positive = data2[data2['Accepted'].isin([1])]
negative = data2[data2['Accepted'].isin([0])]

fig, ax = plt.subplots(figsize=(12, 8))
ax.scatter(positive['Test 1'],
           positive['Test 2'],
           s=50,
           c='b',
           marker='o',
           label='Accepted')
ax.scatter(negative['Test 1'],
           negative['Test 2'],
           s=50,
           c='r',
           marker='x',
           label='Rejected')
ax.legend()
ax.set_xlabel('Test 1 Score')
ax.set_ylabel('Test 2 Score')
plt.show()
```



这个数据看起来可比前一次的复杂得多。特别地，你会注意到其中没有线性决策界限，来良好的分开两类数据。一个方法是用像逻辑回归这样的线性技术来构造从原始特征的多项式中得到的特征。让我们通过创建一组多项式特征入手吧。

```
In [18]: degree = 5
x1 = data2['Test 1']
x2 = data2['Test 2']

data2.insert(3, 'Ones', 1)

for i in range(1, degree):
    for j in range(0, i):
        data2['F' + str(i) + str(j)] = np.power(x1, i-j) * np.power(x2, j)

data2.drop('Test 1', axis=1, inplace=True)
data2.drop('Test 2', axis=1, inplace=True)

data2.head()
```

Out[18]:

	Accepted	Ones	F10	F20	F21	F30	F31	F32	F40
0	1	1	0.051267	0.002628	0.035864	0.000135	0.001839	0.025089	0.000007
1	1	1	-0.092742	0.008601	-0.063523	-0.000798	0.005891	-0.043509	0.000074
2	1	1	-0.213710	0.045672	-0.147941	-0.009761	0.031616	-0.102412	0.002086
3	1	1	-0.375000	0.140625	-0.188321	-0.052734	0.070620	-0.094573	0.019775
4	1	1	-0.513250	0.263426	-0.238990	-0.135203	0.122661	-0.111283	0.069393

现在，我们需要修改第1部分的成本和梯度函数，包括正则化项。首先是成本函数：

regularized cost (正则化代价函数)

$$J(w) = \frac{1}{m} \sum_{i=1}^m [-y^{(i)} \log(h(x^{(i)})) - (1 - y^{(i)}) \log(1 - h(x^{(i)}))] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

```
In [19]: def costReg(w, X, y, learningRate):
w = np.matrix(w)
X = np.matrix(X)
y = np.matrix(y)
first = np.multiply(-y, np.log(sigmoid(X * w.T)))
second = np.multiply((1 - y), np.log(1 - sigmoid(X * w.T)))
reg = (learningRate /
       (2 * len(X))) * np.sum(np.power(w[:, 1:w.shape[1]], 2))
return np.sum(first - second) / len(X) + reg
```

请注意等式中的"reg" 项。还注意到另外的一个“学习率”参数。这是一种超参数，用来控制正则化项。现在我们需要添加正则化梯度函数：

如果我们要使用梯度下降法令这个代价函数最小化，因为我们未对 w_0 进行正则化，所以梯度下降算法将分两种情形：

重复 直到 收敛{

$$w_0 := w_0 - a \frac{1}{m} \sum_{i=1}^m [h(x^{(i)}) - y^{(i)}] x_0^{(i)}$$

$$w_j := w_j - a \frac{1}{m} \sum_{i=1}^m [h(x^{(i)}) - y^{(i)}] x_j^{(i)} + \frac{\lambda}{m} w_j$$

}

```
In [20]: def gradientReg(w, X, y, learningRate):
    w = np.matrix(w)
    X = np.matrix(X)
    y = np.matrix(y)

    parameters = int(w.ravel().shape[1])
    grad = np.zeros(parameters)

    error = sigmoid(X * w.T) - y

    for i in range(parameters):
        term = np.multiply(error, X[:, i])

        if (i == 0):
            grad[i] = np.sum(term) / len(X)
        else:
            grad[i] = (np.sum(term) / len(X)) + (
                learningRate / len(X)) * w[:, i])

    return grad
```

就像在第一部分中做的一样，初始化变量。

```
In [21]: # set X and y (remember from above that we moved the label to column 0)
cols = data2.shape[1]
X2 = data2.iloc[:, 1:cols]
y2 = data2.iloc[:, 0:1]

# convert to numpy arrays and initialize the parameter array w
X2 = np.array(X2.values)
y2 = np.array(y2.values)
w2 = np.zeros(11)
```

让我们初始学习率到一个合理值。如果有必要的话（即如果惩罚太强或不够强），我们可以之后再折腾这个。

```
In [22]: learningRate = 1
```

现在，让我们尝试调用新的默认为0的 w 的正则化函数，以确保计算工作正常。

```
In [23]: costReg(w2, X2, y2, learningRate)
```

```
Out[23]: 0.6931471805599454
```

```
In [24]: gradientReg(w2, X2, y2, learningRate)
```

```
Out[24]: array([0.00847458, 0.01878809, 0.05034464, 0.01150133, 0.01835599,
                0.00732393, 0.00819244, 0.03934862, 0.00223924, 0.01286005,
                0.00309594])
```

现在我们可以使用和第一部分相同的优化函数来计算优化后的结果。

```
In [25]: result2 = opt.fmin_tnc(func=costReg, x0=w2, fprime=gradientReg, args=(X2, y2, learni
result2
```

```
Out[25]: (array([ 0.53010248,  0.29075567, -1.60725764, -0.58213819,  0.01781027,
                -0.21329508, -0.40024142, -1.37144139,  0.02264304, -0.9503358 ,
                0.0344085 ]),
          22,
          1)
```

最后，我们可以使用第1部分中的预测函数来查看我们的方案在训练数据上的准确度。

```
In [26]: w_min = np.matrix(result2[0])
predictions = predict(w_min, X2)
correct = [1 if ((a == 1 and b == 1) or (a == 0 and b == 0)) else 0 for (a, b) in
accuracy = (sum(map(int, correct)) % len(correct))
print ('accuracy = {0}%'.format(accuracy))
```

```
accuracy = 78%
```

虽然我们实现了这些算法，值得注意的是，我们还可以使用高级Python库像scikit-learn来解决这个问题。

```
In [27]: from sklearn import linear_model#调用sklearn的线性回归包
model = linear_model.LogisticRegression(penalty='l2', C=1.0)
model.fit(X2, y2.ravel())
```

```
Out[27]: LogisticRegression()
```

```
In [28]: model.score(X2, y2)
```

```
Out[28]: 0.6610169491525424
```

这个准确度和我们刚刚实现的差了好多，不过请记住这个结果可以使用默认参数下计算的结果。我们可能需要做一些参数的调整来获得和我们之前结果相同的精确度。

参考

[1] Andrew Ng. Machine Learning[EB/OL].

StanfordUniversity,2014.<https://www.coursera.org/course/ml>
(<https://www.coursera.org/course/ml>)

[2] 李航. 统计学习方法[M]. 北京: 清华大学出版社,2019.

[3] 逻辑回归实战练习——根据学生成绩预测是否被录取,<https://zhuanlan.zhihu.com/p/61414743> (<https://zhuanlan.zhihu.com/p/61414743>)

In []: